Teoría de la Computación (INFO1148)

AgroReglas

Diseño e implementación de un analizador sintáctico LL(1)

Fernando Juanfra Vicente

https://github.com/Juanfra272/tc-proyecto1

Octubre de 2026

Índice

Definición y análisis formal **3**

1. Introducción y dominio **3**

2. Especificación del lenguaje **3**

2.1 Alfabeto y contrato léxico 3

2.2 Reglas de formación y significado previsto 4

2.3 Cinco ejemplos válidos 4

3. Gramática libre de contexto **4**

4. Análisis y resolución de conflictos **5**

4.1 Recursión izquierda 5

4.2 Factorización por la izquierda 6

4.3 Ambigüedad y precedencia 6

4.4 Extensión integrada 6

5. Cálculo de FIRST y FOLLOW **6**

6. Construcción de la tabla LL(1) **7**

Producciones y conjuntos SELECT 7

FIRST y FOLLOW completos 8

Tabla predictiva LL(1) 8

Comprobaciones 8

7. Cinco casos inválidos y resultados esperados **9**

8. Validación y alcance de la entrega **9**

Referencias del proyecto **10**

Diseño e implementación del parser **11**

9. Interfaz e integración **11**

10. Relación con la gramática y la tabla LL(1) **11**

11. Construcción y significado del AST **12**

12. Ubicaciones y errores 13

13. Pruebas y resultados 13

14. Evidencia reproducible 14

15. Límites prácticos 14

Integración, ejecución y casos de prueba 15

16. Lexer como módulo 15

17. Ejecución desde texto o archivo 15

18. Casos de prueba 15

19. Automatización y resultados 16

20. Evidencia 16

21. Resultados de los 25 casos 16

Conclusiones 19

Anexos 20

A. Código fuente 20

B. Demostración funcional 20

Definición y análisis formal

Autoría: Fernando.

1. Introducción y dominio

AgroReglas es un lenguaje de dominio específico para expresar reglas de monitoreo agrícola. Cada regla relaciona mediciones o estados de sensores con una acción sobre un dispositivo. Por ejemplo, una humedad baja puede indicar la apertura de una válvula. El proyecto estudia la estructura de esas reglas mediante una gramática libre de contexto y un análisis predictivo LL(1).

Una regla tiene la forma WHEN condición THEN verbo destino;. Un programa contiene una o más reglas. El reconocimiento sintáctico comprueba que la entrada cumple esa forma; no ejecuta dispositivos ni verifica que los sensores o acciones existan. Tampoco comprueba tipos, por lo que una comparación como temperatura > TRUE es sintácticamente válida aunque una futura etapa semántica pudiera rechazarla.

2. Especificación del lenguaje

2.1 Alfabeto y contrato léxico

La definición toma como referencia la función `analizador_lexico` de `anlizador_lexico1.ipynb`, sin modificarla. Los tokens son pares (tipo, lexema). La gramática consume el tipo; conserva el lexema para identificar nombres, valores y el operador relacional concreto.

El alfabeto relevante incluye letras ASCII, guion bajo, dígitos decimales, los signos > < = ! ; ( ) y los espacios permitidos. Los signos = y ! solo forman tokens cuando participan en los operadores listados. Exactamente como en el notebook, NUM usa \d+ de Python y admite dígitos decimales Unicode; ID limita sus letras y dígitos a ASCII. El alfabeto no implica que cualquier combinación de sus caracteres sea válida.

Categoría	Tipo de token	Lexemas o patrón del notebook
Palabras reservadas	WHEN THEN AND OR NOT TRUE FALSE	Los siete nombres, sin distinguir mayúsculas de minúsculas
Identificador	ID	[A-Za-z_][A-Za-z0-9_]*, excepto reservadas
Número	NUM	\d+, entero no negativo sin signo ni punto decimal
Relacional	`OP_REL`	>=, <=, ==, !=, >, <
Fin de regla	`PUNTO_COMA`	;
Agrupación	`PARENT_IZQ`, `PARENT_DER`	(,)
Espacio descartado	No llega al parser	Espacio, tabulador y salto de línea LF, patrón [\t\n]+

Las palabras reservadas se reconocen después de consumir el identificador completo. when produce el tipo WHEN, pero `WHEN_sensor` sigue siendo ID. El lexer conserva la escritura original del lexema. Los nombres se conservan tal como se escriben; no se define equivalencia semántica entre sensor y Sensor.

Se reconocen primero los operadores relacionales de dos caracteres. Un carácter ajeno a las categorías produce ValueError. No hay comentarios, cadenas, números negativos, decimales ni separadores de acciones. El carácter CR \r no se descarta en el lexer actual. La CLI normaliza CRLF y CR a LF antes del análisis, como se documenta en la sección de integración; esta especificación no cambia el notebook.

El marcador $ representa fin de entrada y debe añadirlo el parser después de los tokens. No es un carácter aceptado por el lexer. ε representa la cadena vacía y tampoco es un token.

2.2 Reglas de formación y significado previsto

Una comparación contiene un identificador a la izquierda, un operador relacional y un número o booleano a la derecha. No se admiten comparaciones encadenadas, identificadores como valor derecho ni booleanos aislados. Las condiciones combinan comparaciones mediante AND y OR. La extensión incorpora negación NOT y agrupación con paréntesis, con anidamiento de profundidad finita arbitraria.

La precedencia, de mayor a menor, es agrupación, comparación, NOT, AND, OR. Así, NOT humedad < 30 niega la comparación completa. Para cadenas del mismo operador binario se acuerda asociación a la izquierda en la representación semántica. La gramática usa colas derechas para permitir reconocimiento LL(1). La asociación acordada se obtiene acumulando los operandos en orden. NOT NOT c se interpreta como NOT (NOT c).

La acción consta exactamente de dos identificadores: verbo y destino. abrir valvula y activar alarma son acciones; esos nombres son vocabulario del dominio, no nuevas palabras reservadas. Cada regla termina en punto y coma. El salto de línea solo separa tokens y no termina una regla.

2.3 Cinco ejemplos válidos

Son ejemplos de la especificación y forman parte de los 25 casos ejecutados del parser. Los cinco se aceptan; sus resultados y árboles se registran en `docs/resultados_casos.txt` y `docs/evidencias/ejecucion.txt`.

ID	Entrada	Propiedad ilustrada
V01	WHEN humedad < 30 THEN abrir valvula;	Regla mínima con valor numérico
V02	WHEN lluvia == FALSE THEN activar riego;	Valor booleano
V03	WHEN temperatura >= 28 AND humedad <= 40 OR emergencia == TRUE THEN activar alarma;	AND tiene prioridad sobre OR
V04	WHEN NOT (lluvia == TRUE OR humedad > 60) AND nivel != 0 THEN encender bomba;	Extensión con negación y agrupación
V05	WHEN NOT NOT (humedad < 30 AND (lluvia == FALSE OR reserva > 0)) THEN abrir valvula; WHEN nivel == 0 THEN apagar bomba;	Negación repetida, anidamiento y varias reglas

Se cumplen los mínimos del enunciado: siete palabras reservadas, identificadores, números y booleanos, seis operadores relacionales, dos conectores lógicos, una estructura principal, varias reglas y construcción compuesta.

3. Gramática libre de contexto

Formalmente, G = (N, T, P, S), con S = Programa.

N = {Programa, Resto, Regla, Expr, ColaOR, Termino, ColaAND, Unaria, Comparacion, Valor, Accion}.

T = {WHEN, THEN, OR, AND, NOT, `PARENT_IZQ`, `PARENT_DER`, ID, `OP_REL`, NUM, TRUE, FALSE, `PUNTO_COMA`}.

P contiene las siguientes 18 producciones. La numeración coincide con la tabla predictiva y con el orden de las alternativas en `especificacion/gramatica.json`.

```
1. Programa    → Regla Resto
2. Resto       → Regla Resto
```

```
 3. Resto       → ε
 4. Regla       → WHEN Expr THEN Accion PUNTO_COMA
 5. Expr        → Termino ColaOR
 6. ColaOR      → OR Termino ColaOR
 7. ColaOR      → ε
 8. Termino     → Unaria ColaAND
 9. ColaAND     → AND Unaria ColaAND
10. ColaAND     → ε
11. Unaria      → NOT Unaria
12. Unaria      → PARENT_IZQ Expr PARENT_DER
13. Unaria      → Comparacion
14. Comparacion → ID OP_REL Valor
15. Valor       → NUM
16. Valor       → TRUE
17. Valor       → FALSE
18. Accion      → ID ID
```

Una entrada vacía no pertenece al lenguaje porque Programa exige la primera Regla. Solo Resto, ColaOR y ColaAND son anulables. Las reglas tienen longitud finita y cada repetición no vacía consume tokens.

Una derivación izquierda de la secuencia de tipos de V01 ilustra la conexión con el lexer:

```
Programa
⇒ Regla Resto
⇒ WHEN Expr THEN Accion PUNTO_COMA Resto
⇒ WHEN Termino ColaOR THEN Accion PUNTO_COMA Resto
⇒ WHEN Unaria ColaAND ColaOR THEN Accion PUNTO_COMA Resto
⇒ WHEN Comparacion ColaAND ColaOR THEN Accion PUNTO_COMA Resto
⇒ WHEN ID OP_REL Valor ColaAND ColaOR THEN Accion PUNTO_COMA Resto
⇒ WHEN ID OP_REL NUM ColaAND ColaOR THEN Accion PUNTO_COMA Resto
⇒ WHEN ID OP_REL NUM ColaOR THEN Accion PUNTO_COMA Resto
⇒ WHEN ID OP_REL NUM THEN Accion PUNTO_COMA Resto
⇒ WHEN ID OP_REL NUM THEN ID ID PUNTO_COMA Resto
⇒ WHEN ID OP_REL NUM THEN ID ID PUNTO_COMA
```

4. Análisis y resolución de conflictos

4.1 Recursión izquierda

Una GLC presenta recursión izquierda si existe un no terminal A tal que $A \Rightarrow^{+} A\ \alpha$. Un descenso recursivo directo sobre esa estructura intentaría reconocer A sin consumir entrada y podría no terminar.

Una formulación inicial habitual sería Expr → Expr OR Termino | Termino y Termino → Termino AND Unaria | Unaria. Ambas reglas tienen recursión izquierda inmediata. Sus alternativas también intentarían ocupar las mismas celdas de la tabla: por ejemplo, ambas alternativas de Expr pueden comenzar con ID.

Se aplica la transformación A → A α | β a A → β A', A' → α A' | ε. De ella se obtienen `Expr/ColaOR` y `Termino/ColaAND`, preservando las secuencias de operandos y operadores. La asociación del AST se acuerda por separado, como indica la sección 2.2.

En la versión final no existe recursión izquierda inmediata ni indirecta. Desde Programa o Resto se llega a Regla, cuyo primer símbolo es WHEN. Las rutas iniciales Expr → Termino → Unaria → Comparacion terminan en ID; las demás alternativas de Unaria comienzan en NOT o `PARENT_IZQ`. La vuelta de Unaria a Expr consume antes `PARENT_IZQ`. Las colas recursivas consumen OR o AND antes de repetirse. El verificador comprueba también ciclos a través de prefijos anulables mediante la relación de esquinas izquierdas y no encuentra ninguno.

4.2 Factorización por la izquierda

La propuesta Programa ⟶ Regla Programa | Regla comparte el prefijo Regla. Ambas alternativas tienen FIRST igual a {WHEN}, por lo que entrarían en conflicto en M[Programa, WHEN]. Se factoriza como Programa ⟶ Regla Resto y Resto ⟶ Regla Resto | ε.

La formulación Comparacion ⟶ ID `OP_REL` NUM | ID `OP_REL` TRUE | ID `OP_REL` FALSE también comparte un prefijo. Se factoriza en Comparacion ⟶ ID `OP_REL` Valor y Valor ⟶ NUM | TRUE | FALSE.

La gramática final no necesita más factorización. Las alternativas no vacías de cada no terminal con elección tienen conjuntos FIRST disjuntos. En particular, las tres alternativas de Unaria empiezan respectivamente por NOT, `PARENT_IZQ` e ID.

4.3 Ambigüedad y precedencia

Una gramática es ambigua cuando alguna cadena tiene dos árboles de derivación distintos. Una formulación plana Condicion ⟶ Condicion AND Condicion | Condicion OR Condicion | Comparacion permite que a OR b AND c se agrupe como (a OR b) AND c o como a OR (b AND c), donde a, b y c representan comparaciones completas. También contiene recursión izquierda. Factorizar por sí solo no resuelve esa elección de significado.

La gramática final separa niveles. Expr une términos mediante OR; Termino une unarias mediante AND; Unaria resuelve la negación, la agrupación o la comparación. Por ello, a OR b AND c se interpreta como a OR (b AND c), y NOT a AND b como (NOT a) AND b. (a OR b) AND c requiere los paréntesis escritos explícitamente. NOT se aplica a una sola Unaria, de modo que negar toda una disyunción requiere NOT (a OR b).

Los SELECT disjuntos de la gramática final demuestran que es LL(1). Toda GLC LL(1) es no ambigua: una derivación izquierda queda determinada por el no terminal pendiente y el siguiente tipo de token. Esto justifica la ausencia de ambigüedad para todas sus cadenas, sin depender solo de ejemplos.

4.4 Extensión integrada

El núcleo utiliza Unaria ⟶ Comparacion. La extensión añade Unaria ⟶ NOT Unaria | `PARENT_IZQ` Expr `PARENT_DER`, manteniendo esa alternativa original. Sus dos nuevos tokens iniciales son diferentes de ID, por lo que no introducen conflictos `FIRST/FIRST`.

Con la extensión, FIRST de Unaria, Termino y Expr pasa de {ID} a {ID, NOT, `PARENT_IZQ`}. El cierre de paréntesis añade `PARENT_DER` a FOLLOW de Expr y sus colas y operandos internos. La tabla final incorpora las entradas con NOT y `PARENT_IZQ`, y permite seleccionar las alternativas vacías de las colas ante `PARENT_DER`. No se permite una condición vacía entre paréntesis. V04 y V05 especifican usos válidos; I03 e I04 especifican errores de la extensión.

5. Cálculo de FIRST y FOLLOW

FIRST(α) contiene los terminales que pueden iniciar alguna cadena derivada de α. Incluye ε cuando $\alpha \Rightarrow^* \varepsilon$. Para un terminal t, FIRST(t) = {t}. Al recorrer una secuencia se agregan los FIRST sin ε hasta encontrar un símbolo no anulable; se incluye ε solo si todos son anulables.

FOLLOW(A) contiene los terminales que pueden aparecer inmediatamente después de A en una forma sentencial. Se inicia con $ ∈ FOLLOW(Programa). Para cada producción A ⟶ α B β, se agrega FIRST(β) − {ε} a FOLLOW de B. Cuando β es anulable, se agrega también FOLLOW de A. Se repiten estas operaciones hasta alcanzar un punto fijo. FOLLOW nunca contiene ε.

Por ejemplo, FIRST(Unaria) = {NOT, PARENT_IZQ, ID} por sus tres alternativas. FIRST(ColaAND) = {AND, ε}, mientras que FIRST(Resto) = {WHEN, ε}. Como Termino no es anulable, FIRST(Expr) = FIRST(Termino) = FIRST(Unaria).

De Regla ⟶ WHEN Expr THEN Accion PUNTO_COMA se obtiene THEN ∈ FOLLOW(Expr) y PUNTO_COMA ∈ FOLLOW(Accion). De la agrupación se obtiene PARENT_DER ∈ FOLLOW(Expr). Por tanto, FOLLOW(Expr) = {THEN, PARENT_DER}. Como ColaOR termina la producción de Expr, hereda ese FOLLOW. Termino puede ir seguido por OR, o por FOLLOW de Expr cuando ColaOR ⟹ ε. Análogamente, Unaria añade AND y hereda FOLLOW de Termino. Comparacion y Valor heredan FOLLOW de Unaria.

Las tablas completas, para los once no terminales, están en `first_follow_tabla.txt`. El informe incorpora sus tablas FIRST y FOLLOW junto con la tabla predictiva. Se genera desde la GLC formal del proyecto, para evitar divergencias de transcripción.

6. Construcción de la tabla LL(1)

Para cada producción A ⟶ α, se coloca su número en M[A, t] para todo t ∈ FIRST(α) − {ε}. Si ε ∈ FIRST(α), se coloca además en M[A, t] para todo t ∈ FOLLOW(A). Este conjunto de columnas es SELECT de la producción. Una celda sin producción indica error sintáctico. Una celda con dos producciones distintas indica conflicto LL(1).

La tabla completa de `first_follow_tabla.txt` incluye los trece terminales y $. Tiene 25 celdas ocupadas de 154 y cero conflictos. Por ejemplo, M[Unaria, NOT] = 11, M[Unaria, PARENT_IZQ] = 12, M[Unaria, ID] = 13, M[ColaAND, OR] = 10, M[ColaOR, PARENT_DER] = 7 y M[Resto, $] = 3.

En los tres no terminales anulables, los SELECT son disjuntos:

No terminal	SELECT de alternativa no vacía	SELECT de ε
Resto	{WHEN}	{$}
ColaOR	{OR}	{THEN, PARENT_DER}
ColaAND	{AND}	{OR, THEN, PARENT_DER}

Los demás puntos de elección son Unaria, con {NOT}, {PARENT_IZQ} e {ID}, y Valor, con {NUM}, {TRUE} y {FALSE}. No hay conflictos FIRST/FIRST ni FIRST/FOLLOW. El reconocimiento podrá decidir con un token de anticipación, sin retroceso. Debe aceptar únicamente tras consumir todas las reglas y alcanzar $.

Generado por python `herramientas/validar_gramatica.py --escribir`.

Producciones y conjuntos SELECT

N.º	Producción	SELECT
1	Programa ⟶ Regla Resto	{ WHEN }
2	Resto ⟶ Regla Resto	{ WHEN }
3	Resto ⟶ ε	{ $ }
4	Regla ⟶ WHEN Expr THEN Accion PUNTO_COMA	{ WHEN }
5	Expr ⟶ Termino ColaOR	{ ID, NOT, PARENT_IZQ }
6	ColaOR ⟶ OR Termino ColaOR	{ OR }
7	ColaOR ⟶ ε	{ PARENT_DER, THEN }
8	Termino ⟶ Unaria ColaAND	{ ID, NOT, PARENT_IZQ }
9	ColaAND ⟶ AND Unaria ColaAND	{ AND }

N.º	Producción	SELECT
10	ColaAND → ε	{ OR, PARENT_DER, THEN }
11	Unaria → NOT Unaria	{ NOT }
12	Unaria → PARENT_IZQ Expr PARENT_DER	{ PARENT_IZQ }
13	Unaria → Comparacion	{ ID }
14	Comparacion → ID OP_REL Valor	{ ID }
15	Valor → NUM	{ NUM }
16	Valor → TRUE	{ TRUE }
17	Valor → FALSE	{ FALSE }
18	Accion → ID ID	{ ID }

FIRST y FOLLOW completos

No terminal	FIRST	FOLLOW
Programa	{ WHEN }	{ $ }
Resto	{ WHEN, ε }	{ $ }
Regla	{ WHEN }	{ $, WHEN }
Expr	{ ID, NOT, PARENT_IZQ }	{ PARENT_DER, THEN }
ColaOR	{ OR, ε }	{ PARENT_DER, THEN }
Termino	{ ID, NOT, PARENT_IZQ }	{ OR, PARENT_DER, THEN }
ColaAND	{ AND, ε }	{ OR, PARENT_DER, THEN }
Unaria	{ ID, NOT, PARENT_IZQ }	{ AND, OR, PARENT_DER, THEN }
Comparacion	{ ID }	{ AND, OR, PARENT_DER, THEN }
Valor	{ FALSE, NUM, TRUE }	{ AND, OR, PARENT_DER, THEN }
Accion	{ ID }	{ PUNTO_COMA }

Tabla predictiva LL(1)

Cada número remite a una producción. error representa una celda vacía.

No terminal	WHEN	THEN	OR	AND	NOT	PARENT_IZQ	PARENT_DER	ID	OP_REL	NUM	TRUE	FALSE	PUNTO_COMA	$
Programa	1	error	error	error	error	error	error	error	error	error	error	error	error	error
Resto	2	error	error	error	error	error	error	error	error	error	error	error	error	3
Regla	4	error	error	error	error	error	error	error	error	error	error	error	error	error
Expr	error	error	error	error	5	5	error	5	error	error	error	error	error	error
ColaOR	error	7	6	error	error	error	7	error	error	error	error	error	error	error
Termino	error	error	error	error	8	8	error	8	error	error	error	error	error	error
ColaAND	error	10	10	9	error	error	10	error	error	error	error	error	error	error
Unaria	error	error	error	error	11	12	error	13	error	error	error	error	error	error
Comparacion	error	error	error	error	error	error	error	14	error	error	error	error	error	error
Valor	error	error	error	error	error	error	error	error	error	15	16	17	error	error
Accion	error	error	error	error	error	error	error	18	error	error	error	error	error	error

Comprobaciones

- 11 no terminales, 13 terminales y 18 producciones.

- 25 celdas ocupadas de 154.
- Conflictos LL(1): 0.
- No terminales con recursión izquierda: { }.
- No terminales inaccesibles: { }.
- No terminales improductivos: { }.

7. Cinco casos inválidos y resultados esperados

Los cinco casos son errores sintácticos con tokens reconocibles por el lexer. Las ubicaciones usan línea y columna desde 1; el desplazamiento absoluto de `casos.json` cuenta caracteres desde 0. EOF se ubica inmediatamente después del último token, sin contar los espacios finales; una entrada sin tokens se ubica en 1:1. Se especifica el primer punto donde ya no puede continuar una derivación válida, sin consumir otro token para intentar reparar la entrada.

ID	Entrada	Línea:columna	Resultado esperado
I01	WHEN humedad < THEN abrir valvula;	1:16	Rechazar en THEN: falta NUM, TRUE o FALSE después de <
I02	WHEN humedad < 30 AND THEN abrir valvula;	1:23	Rechazar en THEN: después de AND se esperaba ID, NOT o (
I03	WHEN () THEN abrir valvula;	1:7	Rechazar en): se esperaba ID, NOT o (para iniciar la condición
I04	WHEN (humedad < 30 THEN abrir valvula;	1:20	Rechazar en THEN: falta) para cerrar el grupo
I05	WHEN humedad < 30 THEN abrir valvula	1:37	Rechazar en EOF: falta ; para terminar la regla

I01 falla en la fila Valor; I02 en Unaria; I03 en Expr. En I04, las colas internas pueden derivar ε ante THEN, pero el siguiente terminal obligatorio es `PARENT_DER`, por lo que allí se detecta la falta del cierre. I05 falla al cotejar el terminal `PUNTO_COMA` con $. Los mensajes de error identifican la causa y la ubicación del primer error.

El JSON incluye los cinco ejemplos válidos y estos cinco inválidos, sus tipos de tokens, ubicaciones y resultados esperados. El campo `resultado_esperado` describe el comportamiento definido para cada caso; los resultados obtenidos se contrastan en `docs/resultados_casos.txt`.

8. Validación y alcance de la entrega

Ejecutar desde la raíz:

```
python herramientas/validar_gramatica.py
```

El verificador independiente usa solo la biblioteca estándar de Python. Calcula FIRST, FOLLOW y SELECT por punto fijo, construye la tabla, detecta colisiones, comprueba recursión izquierda incluyendo prefijos anulables, accesibilidad y productividad. Además compara las tablas publicadas con el resultado calculado. Sus autocontroles usan gramáticas pequeñas con resultados conocidos, incluyendo un conflicto `FIRST/FIRST`, uno `FIRST/FOLLOW` y recursión izquierda indirecta.

Para regenerar las tablas después de un cambio deliberado en la gramática:

```
python herramientas/validar_gramatica.py --escribir
```

La validación estática obtiene 18 producciones, 25 celdas ocupadas, cero conflictos, cero no terminales con recursión izquierda y ningún no terminal inaccesible o improductivo. El verificador comprueba las propiedades de la gramática; el parser implementado en `agroreglas/parser.py` reconoce los programas AgroReglas. No se utiliza un generador de parsers.

La validación formal es coherente con el lexer existente. La ejecución de los 25 casos coincide con sus resultados esperados: se aceptan los diez casos V, los tres casos L aceptados y se rechazan los diez casos I y los dos casos L inválidos. La evidencia está en `docs/resultados_casos.txt` y `docs/evidencias/ejecucion.txt`.

Referencias del proyecto

- Proyecto `Analisis_Sintactico.pdf`, secciones 5, 6.1 a 6.4, 6.8 y 6.9: mínimos del lenguaje, definición formal, análisis LL(1), pruebas y extensión.
- `Rubrica_Proyecto.pdf`, criterios 1 a 4 y 7: coherencia del lenguaje, GLC, `FIRST/FOLLOW`, tabla y extensión integrada.
- `anlizador_lexico1.ipynb`, primera celda: interfaz y categorías del lexer proporcionado.

Diseño e implementación del parser

Autoría: Juanfra.

9. Interfaz e integración

El paquete agroreglas implementa un parser predictivo propio en Python, sin generadores de parsers ni dependencias externas. La función pública es:

```
from agroreglas import analizar_tokens, ErrorSintactico
ast = analizar_tokens(tokens, fuente)
```

Los tokens son los pares (tipo, lexema) que entrega `analizador_lexico`, extraído de `anlizador_lexico1.ipynb` a `agroreglas/lexer.py`. fuente debe ser el mismo texto exacto que recibió el lexer. El parser añade el marcador $; el llamador no debe incluirlo. El resultado es un diccionario que representa el programa completo. Si la entrada es sintácticamente inválida, se lanza ErrorSintactico. Si los lexemas no corresponden al texto proporcionado, se lanza ValueError por incumplimiento del contrato de integración.

`analizar_programa`(fuente) aplica el lexer y el parser en un solo paso. La ejecución desde archivo o texto se describe en la sección de integración y en el README.

10. Relación con la gramática y la tabla LL(1)

`parser.py` lee `especificacion/gramatica.json` y utiliza el cálculo estático analizar de `herramientas/validar_gramatica.py` para obtener la tabla documentada por Fernando. Esta función calcula FIRST, FOLLOW y SELECT; no reconoce el programa ni genera el código del parser. El reconocimiento y las acciones semánticas están escritos en `agroreglas/parser.py`. La carga rechaza una gramática con conflictos LL(1) o recursión izquierda.

La pila comienza con $ y Programa, con Programa en la cima. En cada paso se consulta el tipo del token actual:

- Si la cima es un terminal, debe coincidir con ese tipo. Se consume un token.
- Si es un no terminal A, se consulta M[A, tipo]. Se sustituye A por el lado derecho de la producción, apilado en orden inverso.
- Si la producción deriva epsilon, no se agregan símbolos de entrada.
- Si falta la celda de tabla o no coincide un terminal, se informa el primer error y se detiene el reconocimiento.

Solo se acepta después de reconocer Programa y cotejar $ con el fin real de los tokens. No se aceptan programas vacíos ni tokens sobrantes después de una regla. No hay retroceso ni recuperación automática de errores.

Cada expansión deja además una marca de reducción con el nombre del no terminal y su cantidad de hijos. Una segunda pila almacena tokens reconocidos y valores semánticos. Al alcanzar una marca, se retiran sus hijos, se aplica `_reducir` y se apila el resultado. El mismo recorrido reconoce la entrada y construye el AST.

Las pilas son listas de Python; el reconocimiento no usa recursión de funciones. Resto y las colas lógicas se representan temporalmente con pares enlazados para evitar copiar listas de tamaño creciente. El reconocimiento es lineal en el número de tokens para esta gramática fija, sin contar el coste de convertir los números; el almacenamiento es lineal en el tamaño de la entrada y del AST.

11. Construcción y significado del AST

La representación utiliza estos nodos:

- Programa: lista ordenada de reglas.
- Regla: condicion y accion.
- Comparacion: identificador, operador relacional concreto y valor.
- Numero: valor entero de Python.
- Booleano: valor True o False de Python, presentado como TRUE o FALSE en el árbol.
- Logica: operador AND u OR, izquierda y derecha.
- Negacion: operando de NOT.
- Accion: verbo y destino conservados como identificadores.

Las producciones 1 a 3 reúnen las reglas; la 4 crea Regla. Las producciones 5 a 10 acumulan las operaciones lógicas. Las 11 a 13 implementan negación, agrupación y comparación. La 14 crea Comparacion; las 15 a 17 convierten el valor; la 18 crea Accion.

Termino procesa AND dentro de Expr, de modo que AND tiene mayor prioridad que OR. Cada cola se recorre de izquierda a derecha para producir asociación izquierda, aunque la gramática esté escrita con recursión derecha. Unaria construye NOT sobre su operando completo y permite negaciones repetidas. Los paréntesis devuelven el AST de la expresión interior; se omiten como nodos porque la agrupación ya queda expresada en las relaciones entre hijos.

Por ejemplo, V04 produce AND(NOT(OR(lluvia == TRUE, humedad > 60)), nivel != 0), seguido por la acción encender bomba. El AST completo está en `ejemplos/arbol_v04.json`. Su versión jerárquica está en `ejemplos/arbol_v04.txt` y puede regenerarse con `formatear_arbol` de `agroreglas/arbol.py`. Esta presentación también usa una pila para recorrer el árbol.

El parser comprueba sintaxis. No evalúa condiciones, no ejecuta acciones ni valida la existencia de sensores o dispositivos. Tampoco realiza comprobación semántica de tipos.

AST del caso V04 producido por `formatear_arbol`:

```
WHEN NOT (lluvia == TRUE OR humedad > 60) AND nivel != 0 THEN encender bomba;

Programa
  Regla
    AND
      NOT
        OR
          Comparacion lluvia ==
            Booleano TRUE
          Comparacion humedad >
            Numero 60
```

```
    Comparacion nivel !=
      Numero 0
  Accion encender bomba
```

12. Ubicaciones y errores

El lexer devuelve pares sin posiciones. `_ubicar` recorre el texto original en orden, descarta únicamente espacio, tabulador y LF, y comprueba cada lexema en su posición. Así el parser no depende de cambiar las reglas léxicas del docente. No busca lexemas con find, lo que evitaría detectar texto omitido. Los tipos se toman del lexer, que es responsable de clasificarlos correctamente.

El desplazamiento cuenta caracteres Unicode desde cero. Línea y columna parten de uno. Un tabulador cuenta como un carácter, no como varias columnas visuales. EOF se ubica inmediatamente después del último token, sin contar los espacios finales. Una entrada sin tokens se ubica en línea 1, columna 1. El contrato léxico no admite CR; la lectura de archivos lo convierte en LF, como se describe en la sección de integración.

ErrorSintactico conserva el token, el símbolo pendiente y los tipos esperados. `como_dict` devuelve estos datos para automatizar las comprobaciones. El mensaje incluye causa, lexema encontrado, línea, columna y desplazamiento. Para una fila vacía de la tabla, los tipos esperados son sus columnas ocupadas; para un terminal obligatorio, se informa ese terminal. Se descarta el cierre de grupo entre las alternativas cuando no hay un paréntesis abierto. Los tipos se presentan en un orden explícito: número, TRUE y FALSE para valores. El mensaje traduce los tipos internos a nombres legibles y reserva el símbolo pendiente para `como_dict`. Por ejemplo, la falta de ; en I05 se informa en línea 1, columna 37, ante fin de entrada. Los cinco errores de Fernando coinciden exactamente con las posiciones y símbolos de `especificacion/casos.json`.

Los caracteres desconocidos son errores léxicos. El módulo del lexer los informa con ErrorLexico, una subclase de ValueError ubicada con el mismo criterio.

13. Pruebas y resultados

Ejecutar:

```
python -m unittest discover -s pruebas -v
python herramientas/validar_gramatica.py
```

Las cinco pruebas V01 a V05 comparan el AST completo con estructuras esperadas construidas de manera independiente. También comparan los tokens reales del lexer con los publicados en la especificación.

V01 verifica la regla numérica; V02, FALSE; V03, AND antes de OR y TRUE; V04, NOT sobre un grupo y su combinación con AND; V05, negación repetida, paréntesis anidados y dos reglas en orden.

La suite también comprueba asociación izquierda de ambos conectores, errores en varias líneas con dígitos Unicode, entrada vacía, texto sobrante, EOF tras espacios y desacuerdos entre tokens y fuente. Una prueba acotada comprueba 1100 grupos y 1100 reglas para verificar que el parser no depende del límite de recursión de Python.

Los cinco casos inválidos de Fernando se comprueban junto con los 25 casos en `pruebas/test_casos.py`, descrito en la sección de integración. La verificación estática obtiene 18 producciones, 25 celdas ocupadas y cero conflictos LL(1).

14. Evidencia reproducible

Para regenerar las salidas, el AST y las páginas de evidencia:

```
python -m herramientas.evidencia_parser
```

Este comando ejecuta los 25 casos y la suite, y falla si esta última no pasa. Guarda en `docs/evidencias` las salidas de texto y las páginas HTML de la ejecución, del árbol y de la línea de comandos, junto con la tabla `docs/resultados_casos.txt` y los dos archivos del árbol en ejemplos.

Las capturas PNG de `docs/evidencias` muestran esas páginas HTML con salida real de Python. Las páginas se generan escapando la salida, sin modificar los resultados.

15. Límites prácticos

La profundidad del reconocimiento no depende de la pila de llamadas de Python. Las restricciones prácticas son la memoria disponible y las conversiones o serializadores de Python: por ejemplo, la conversión de enteros decimales extremadamente largos puede estar limitada por la configuración del intérprete, y json.dumps puede alcanzar su límite de recursión al exportar un AST con muchísimas negaciones. La presentación jerárquica usa recorrido iterativo. No se cambia la configuración global del intérprete.

Integración, ejecución y casos de prueba

Autoría: Vicente.

16. Lexer como módulo

`agroreglas/lexer.py` contiene la función `analizador_lexico` de `anlizador_lexico1.ipynb` con las mismas expresiones regulares, el mismo orden de categorías y las mismas palabras reservadas. El notebook queda en el repositorio como referencia del docente; el paquete no depende de él.

El único cambio es el error. Un carácter fuera del alfabeto lanza ErrorLexico, una subclase de ValueError, por lo que el código que esperaba ValueError sigue funcionando. ErrorLexico informa el desplazamiento, la línea y la columna con el mismo criterio que ErrorSintactico: desplazamiento desde 0 y línea y columna desde 1. Para comprobar la equivalencia se compararon los tokens del módulo y los del notebook en las 25 entradas de prueba y en cuatro entradas adicionales con dígitos Unicode, CR, un símbolo desconocido y saltos con tabulador. Ambos produjeron los mismos tokens y rechazaron las mismas entradas.

17. Ejecución desde texto o archivo

La función `analizar_programa`(fuente) del paquete une el lexer y el parser: devuelve el AST o lanza el primer ErrorLexico o ErrorSintactico. La línea de comandos usa esa función:

```
python -m agroreglas ejemplos/riego.agro
python -m agroreglas --texto "WHEN humedad < 30 THEN abrir valvula;"
python -m agroreglas ejemplos/riego.agro --json
```

Un archivo de entrada es texto UTF-8 con una o más reglas. Los saltos de línea y los espacios solo separan tokens. Antes de analizar, el programa descarta el BOM inicial y convierte CRLF y CR en LF, tanto en archivos como en `--texto`. Así, un archivo guardado en Windows se analiza igual que uno guardado en Linux, sin cambiar el contrato léxico del notebook, que no admite CR.

Si el programa es válido, se imprime ACEPTADO, la cantidad de reglas y el AST como árbol jerárquico de `formatear_arbol`. Con `--json` se imprime el mismo AST en JSON. Si es inválido, se imprime RECHAZADO, el mensaje del error y la línea de entrada con un ^ bajo la columna del problema. Los tabuladores de esa línea se copian en el margen del ^ para que quede alineado. El código de salida es 0 al aceptar, 1 al rechazar y 2 si el archivo no se puede leer.

18. Casos de prueba

`especificacion/casos.json` reúne 25 entradas distintas: los diez casos de Fernando (V01 a V05 e I01 a I05), cinco válidos nuevos (V06 a V10), cinco inválidos nuevos (I06 a I10) y cinco casos límite (L01 a L05). Cada caso declara su resultado esperado. Cada rechazo declara además la etapa del error, su ubicación, el token encontrado y, si es sintáctico, el símbolo pendiente y los tipos esperados. Estos datos se escribieron a partir de la gramática y de la tabla LL(1), antes de ejecutar el parser.

Los válidos nuevos cubren lo que V01 a V05 no prueban: cadenas de tres AND, cadenas de tres OR, NOT sin paréntesis seguido de AND, dos grupos en una condición y un programa de tres reglas en líneas distintas. V08 confirma que NOT humedad == TRUE AND ... niega solo la primera comparación, como establece la precedencia de la sección 4.3 de Fernando.

Los inválidos nuevos fallan en filas o terminales distintos de los de I01 a I05:

ID	Error	Símbolo pendiente
I06	Falta el operador relacional	OP_REL
I07	Falta THEN	ColaAND
I08	La acción no tiene destino	ID
I09	La regla no empieza con WHEN	Programa
I10	Paréntesis de cierre sin apertura	THEN

Los casos límite prueban los bordes del lenguaje: una entrada con solo espacios (L01), palabras reservadas en minúsculas (L02), negación triple con paréntesis redundantes (L03), identificadores que empiezan con WHEN o NOT y números con ceros a la izquierda (L04), y un número decimal que el lexer rechaza en el punto (L05). Las pruebas de Juanfra completan estos casos con 1100 paréntesis anidados, 1100 reglas, texto sobrante después de una regla y errores en varias líneas.

19. Automatización y resultados

```
python -m unittest discover -s pruebas -v
```

`pruebas/test_casos.py` recorre los 25 casos. En cada uno compara el resultado obtenido con el esperado y, si hay rechazo, compara la etapa, la ubicación, el token y los tipos esperados. En los diez casos de Fernando compara también los tokens del lexer con los publicados. Otras seis pruebas ejecutan la línea de comandos con texto, con texto CRLF, con texto con BOM inicial, con un archivo con BOM y CRLF que contiene un error en la segunda línea, con un error léxico y con un archivo inexistente. `pruebas/test_parser.py` conserva las pruebas de Juanfra que comparan el AST completo de V01 a V05.

Para comprobar que las pruebas detectan fallos, se cambió una columna esperada en `casos.json`: la prueba falló en ese caso y volvió a pasar al restaurar el valor.

Resultado obtenido: 23 métodos de prueba aprobados y 25 de 25 casos con el resultado esperado. La tabla completa con entrada, resultado esperado, resultado obtenido y observación está en `docs/resultados_casos.txt`.

20. Evidencia

```
python -m herramientas.evidencia_parser
```

El comando ejecuta los 25 casos y la suite, y se detiene si alguna prueba falla. Regenera `docs/resultados_casos.txt`, las salidas de texto y las páginas HTML de `docs/evidencias`, incluida `cli.html` con la ejecución desde archivo y desde texto. La salida de los 25 casos se divide en `ejecucion.html` y ejecucion-`nuevos.html` para que cada captura sea legible. Las capturas PNG se toman de esas páginas, por lo que muestran la salida real de Python.

21. Resultados de los 25 casos

Generado por python -m `herramientas.evidencia_parser`. Cada rechazo indica la línea y la columna del primer error. Todos son sintácticos salvo L05, que es léxico. Los mensajes completos están en `docs/evidencias/ejecucion.txt`.

ID	Entrada	Esperado	Obtenido	Observación
V01	`WHEN humedad < 30 THEN abrir valvula;`	aceptar	aceptar	Regla mínima numérica
V02	`WHEN lluvia == FALSE THEN activar riego;`	aceptar	aceptar	Valor booleano
V03	`WHEN temperatura >= 28 AND humedad <= 40 OR emergencia == TRUE THEN activar alarma;`	aceptar	aceptar	Precedencia AND sobre OR
V04	`WHEN NOT (lluvia == TRUE OR humedad > 60) AND nivel != 0 THEN encender bomba;`	aceptar	aceptar	Negación y agrupación
V05	`WHEN NOT NOT (humedad < 30 AND (lluvia == FALSE OR reserva > 0)) THEN abrir valvula; WHEN nivel == 0 THEN apagar bomba;`	aceptar	aceptar	Negación repetida, anidamiento y varias reglas
I01	`WHEN humedad < THEN abrir valvula;`	rechazar	rechazar, 1:16	Falta el valor después del operador relacional
I02	`WHEN humedad < 30 AND THEN abrir valvula;`	rechazar	rechazar, 1:23	Falta la condición después de AND
I03	`WHEN () THEN abrir valvula;`	rechazar	rechazar, 1:7	Grupo vacío
I04	`WHEN (humedad < 30 THEN abrir valvula;`	rechazar	rechazar, 1:20	Falta el paréntesis de cierre
I05	`WHEN humedad < 30 THEN abrir valvula`	rechazar	rechazar, 1:37	Falta el punto y coma final
V06	`WHEN temperatura > 35 AND radiacion >= 800 AND viento <= 20 THEN desplegar malla;`	aceptar	aceptar	AND encadenado con asociación izquierda
V07	`WHEN plaga == TRUE OR hongo == TRUE OR humedad > 85 THEN notificar agronomo;`	aceptar	aceptar	OR encadenado con valores booleanos y numéricos
V08	`WHEN NOT helada == TRUE AND temperatura > 2 THEN abrir invernadero;`	aceptar	aceptar	NOT sin paréntesis niega solo la primera comparación
V09	`WHEN (ph < 5 OR ph > 8) AND (riego_activo == FALSE) THEN corregir ph;`	aceptar	aceptar	Dos grupos, uno con una sola comparación
V10	`WHEN hora == 6 THEN iniciar riego;\nWHEN hora == 8 THEN detener riego;\nWHEN tanque < 10 THEN llenar tanque;`	aceptar	aceptar	Tres reglas en líneas distintas
I06	`WHEN humedad 30 THEN abrir valvula;`	rechazar	rechazar, 1:14	Falta el operador relacional
I07	`WHEN humedad < 30 abrir valvula;`	rechazar	rechazar, 1:19	Falta THEN entre la condición y la acción
I08	`WHEN humedad < 30 THEN abrir;`	rechazar	rechazar, 1:29	Acción incompleta: falta el destino
I09	`humedad < 30 THEN abrir valvula;`	rechazar	rechazar, 1:1	La regla no comienza con WHEN
I10	`WHEN humedad < 30) THEN abrir valvula;`	rechazar	rechazar, 1:18	Paréntesis de cierre sin apertura
L01	`\n\t`	rechazar	rechazar, 1:1	Entrada sin reglas, solo espacios
L02	`when Humedad < 30 then Abrir Valvula;`	aceptar	aceptar	Palabras reservadas en minúsculas y nombres con mayúsculas

ID	Entrada	Esperado	Obtenido	Observación
L03	`WHEN NOT NOT NOT ((((nivel == 0)))) THEN apagar bomba;`	aceptar	aceptar	Negación triple y cuatro paréntesis redundantes
L04	`WHEN WHEN_sensor >= 007 AND not_listo == 0 THEN abrir valvula;`	aceptar	aceptar	Identificadores que empiezan con palabras reservadas y ceros a la izquierda
L05	`WHEN humedad < 3.5 THEN abrir valvula;`	rechazar	rechazar, 1:17	Número decimal: el punto no pertenece al alfabeto

Coinciden 25 de 25 resultados con lo esperado.

Conclusiones

AgroReglas cumple los mínimos del enunciado con siete palabras reservadas, seis operadores relacionales, dos conectores lógicos, varias reglas por programa y una extensión de negación y agrupación que forma parte de la gramática, la tabla, el parser y las pruebas.

La gramática final no tiene recursión izquierda ni prefijos comunes, y la tabla predictiva tiene 25 celdas ocupadas y ningún conflicto. Por eso el parser decide cada paso con un solo token de anticipación, sin retroceso. El verificador vuelve a calcular FIRST, FOLLOW y la tabla desde `especificacion/gramatica.json`, y el parser carga esa misma tabla, así que la teoría documentada y el código no pueden divergir sin que falle la verificación.

El parser usa una pila explícita en lugar de recursión, por lo que una entrada con 1100 paréntesis anidados se analiza sin alcanzar el límite de recursión de Python. Construye el AST en el mismo recorrido mediante reducciones y respeta la precedencia y la asociación izquierda acordadas.

Las 25 entradas documentadas obtienen el resultado esperado y cada rechazo informa la línea, la columna, el token encontrado y lo que se esperaba. La ejecución desde archivo o texto señala la posición del error en la línea original.

El parser solo comprueba sintaxis: no evalúa condiciones ni verifica que los sensores o dispositivos existan, y acepta comparaciones como `temperatura > TRUE`. Una etapa semántica posterior podría validar tipos y vocabulario del dominio sobre el AST que ya se construye.

Anexos

A. Código fuente

El código fuente, la especificación, las pruebas, el README y este informe están en `https://github.com/Juanfra272/tc-proyecto1`. El README indica los requisitos, el formato de entrada y los comandos de ejecución.

B. Demostración funcional

Las capturas muestran páginas generadas por `python -m herramientas.evidencia_parser` con la salida real de Python.

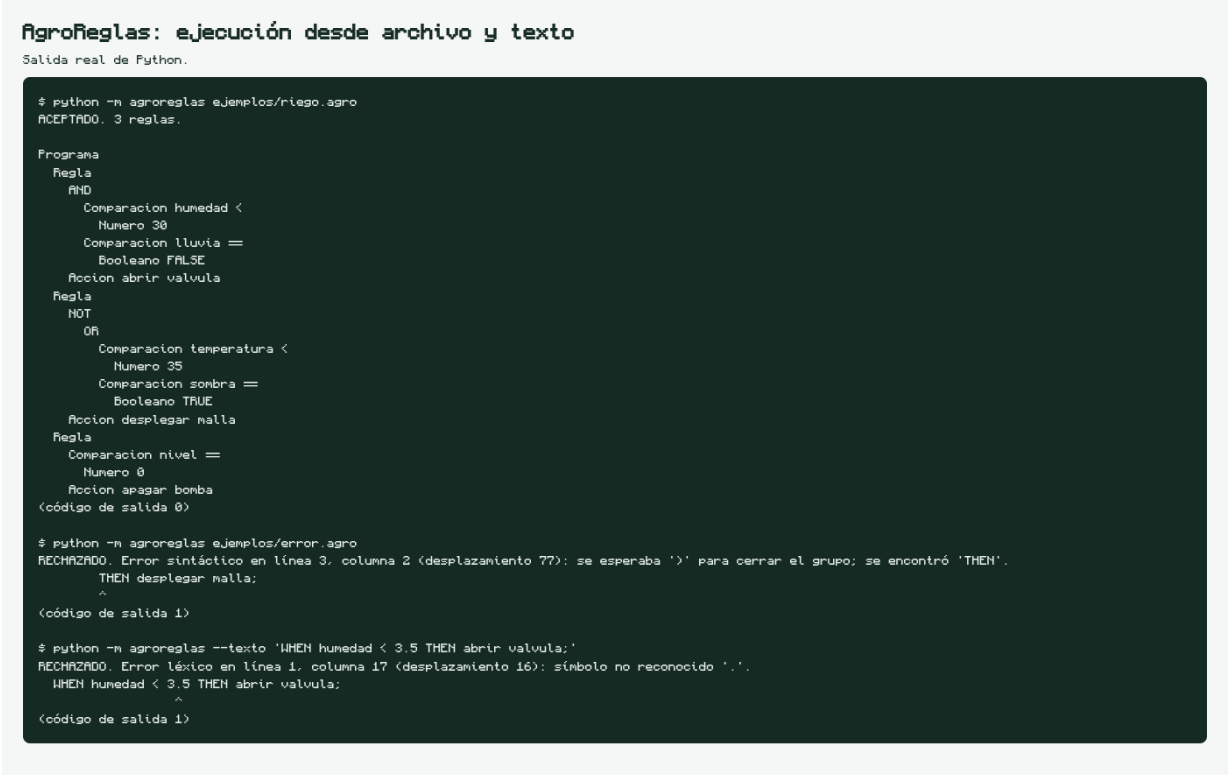


Figura 1: Ejecución desde archivo y desde texto: un programa aceptado con su AST, un error sintáctico en la tercera línea y un error léxico.

AgroReglas: AST de V04

Salida real de Python.

```
WHEN NOT (lluvia == TRUE OR humedad > 60) AND nivel ≠ 0 THEN encender bomba;

Programa
  Regla
    AND
      NOT
        OR
          Comparacion lluvia ==
            Booleano TRUE
          Comparacion humedad >
            Numero 60
        Comparacion nivel ≠
          Numero 0
    Accion encender bomba
```

Figura 2: AST del caso V04, con la negación sobre el grupo y el AND como raíz de la condición.

AgroReglas: casos V01 a I05

Salida real de Python.

```
$ python -m herramientas.demo_parser
V01: 'WHEN humedad < 30 THEN abrir valvula;'
ACEPTADO. 1 regla en el AST.

V02: 'WHEN lluvia == FALSE THEN activar riego;'
ACEPTADO. 1 regla en el AST.

V03: 'WHEN temperatura ≥ 28 AND humedad ≤ 40 OR emergencia == TRUE THEN activar alarma;'
ACEPTADO. 1 regla en el AST.

V04: 'WHEN NOT (lluvia == TRUE OR humedad > 60) AND nivel ≠ 0 THEN encender bomba;'
ACEPTADO. 1 regla en el AST.

V05: 'WHEN NOT NOT (humedad < 30 AND (lluvia == FALSE OR reserva > 0)) THEN abrir valvula; WHEN nivel == 0 THEN apagar bomba;'
ACEPTADO. 2 reglas en el AST.

I01: 'WHEN humedad < THEN abrir valvula;'
RECHAZADO. Error sintáctico en línea 1, columna 16 (desplazamiento 15): se esperaba un número, TRUE, FALSE; se encontró 'THEN'.

I02: 'WHEN humedad < 30 AND THEN abrir valvula;'
RECHAZADO. Error sintáctico en línea 1, columna 23 (desplazamiento 22): se esperaba un identificador, '(', NOT; se encontró 'THEN'.

I03: 'WHEN () THEN abrir valvula;'
RECHAZADO. Error sintáctico en línea 1, columna 7 (desplazamiento 6): se esperaba un identificador, '(', NOT; se encontró ')'.

I04: 'WHEN (humedad < 30 THEN abrir valvula;'
RECHAZADO. Error sintáctico en línea 1, columna 20 (desplazamiento 19): se esperaba ')' para cerrar el grupo; se encontró 'THEN'.

I05: 'WHEN humedad < 30 THEN abrir valvula'
RECHAZADO. Error sintáctico en línea 1, columna 37 (desplazamiento 36): se esperaba ';' al final de la regla; se encontró fin de entrada.
```

Figura 3: Decisión del analizador en los diez casos de Fernando, con el mensaje completo de cada rechazo.



Figura 4: Decisión del analizador en los quince casos nuevos, incluidos los cinco casos límite.